

Intelligent Course Discovery System

Detailed Implementation Plan

Kavini, based on the problem statement, your staff's suggestions, the two datasets, and the additional features we've discussed, I recommend building this as a **modular, explainable, hybrid course recommendation system with an interactive React frontend**.

The goal is to create something that is technically sound, useful to students, visually polished, and demonstrable within your 10-minute presentation.

We'll plan the complete implementation before writing code. This will also give us a clear order of work so we don't end up building features that need to be rewritten later.

1. Project overview

Project title

University Course Finder – Intelligent Course Discovery System

Main objective

Develop a system that understands students' learning goals, identifies relevant courses, analyzes their existing skills, and generates personalized learning paths.

The system should go beyond conventional keyword-based search by combining lexical and semantic retrieval with personalized recommendations.

Core capabilities

Our application will have seven main capabilities:

1. **Intelligent course search:** Find courses using natural-language queries.
2. **Hybrid recommendations:** Combine BM25 and semantic search to retrieve relevant courses.
3. **Explainable recommendations:** Explain why a course is suitable or potentially unsuitable.
4. **Skill gap analysis:** Identify the skills students need to acquire.

5. **Personalized learning paths:** Organize courses into a logical sequence.
6. **Interactive skill visualization:** Let students explore their skills and learning paths.
7. **Evaluation and feedback:** Measure recommendation quality and collect user feedback.

We'll also implement the What-If Skill Toggle and a reliable API fallback mechanism.

2. Final proposed technology stack

Since you're already comfortable with React, I recommend using it rather than learning Streamlit for this project.

Here's our proposed stack.

Component	Technology	Reason
Frontend	React	Interactive, component-based UI
Styling	Tailwind CSS - not sure	Responsive and customizable design
UI components	shadcn/ui	Consistent, polished interface
Graph visualization	React Flow	Interactive skill dependency graph
Charts	Recharts	Evaluation metrics and visualizations
Backend	Python + FastAPI	Modular REST API and microservice
Data processing	Pandas	Dataset cleaning and transformation
Keyword retrieval	BM25	Lexical course search
Semantic retrieval	Sentence Transformers	Natural-language understanding
Similarity	Cosine similarity	Semantic relevance calculation
Hybrid fusion	Reciprocal Rank Fusion (RRF)	Combine retrieval rankings
Recommendation logic	Python	Personalized ranking and explanations

Database	SQLite	Store feedback and saved courses
Embedding storage	NumPy	Store and retrieve course embeddings
Testing	Pytest	Backend and recommendation testing
API testing	Postman/Swagger ui	Test REST endpoints
Version control	Git and GitHub	Track code and collaborate
Deployment	To be decided	Depends on available resources

Why use RRF?

We previously discussed combining BM25 and semantic scores using a weighted formula.

However, their scores have different scales. Normalizing them and selecting suitable weights introduces another tuning problem.

Instead, I suggest we initially use **Reciprocal Rank Fusion (RRF)**.

RRF combines the positions of documents in different search result lists rather than directly adding their raw scores.

For example:

Course	BM25 rank	Semantic rank
Course A	1	4
Course B	3	1
Course C	2	2
Course D	5	3

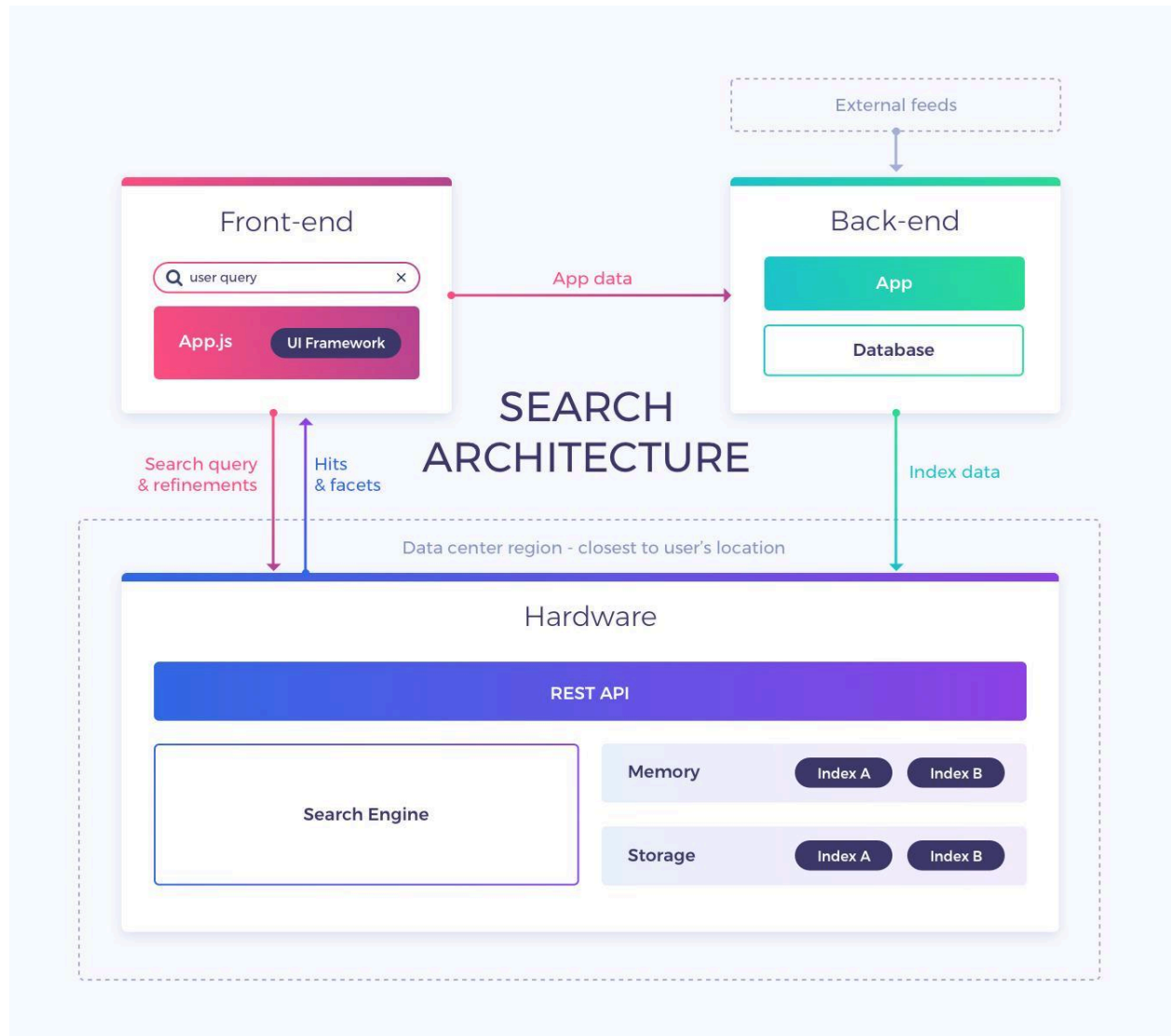
RRF rewards courses that appear near the top of both lists.

We can compare RRF with a weighted-score approach during evaluation and retain whichever works better on our test queries.

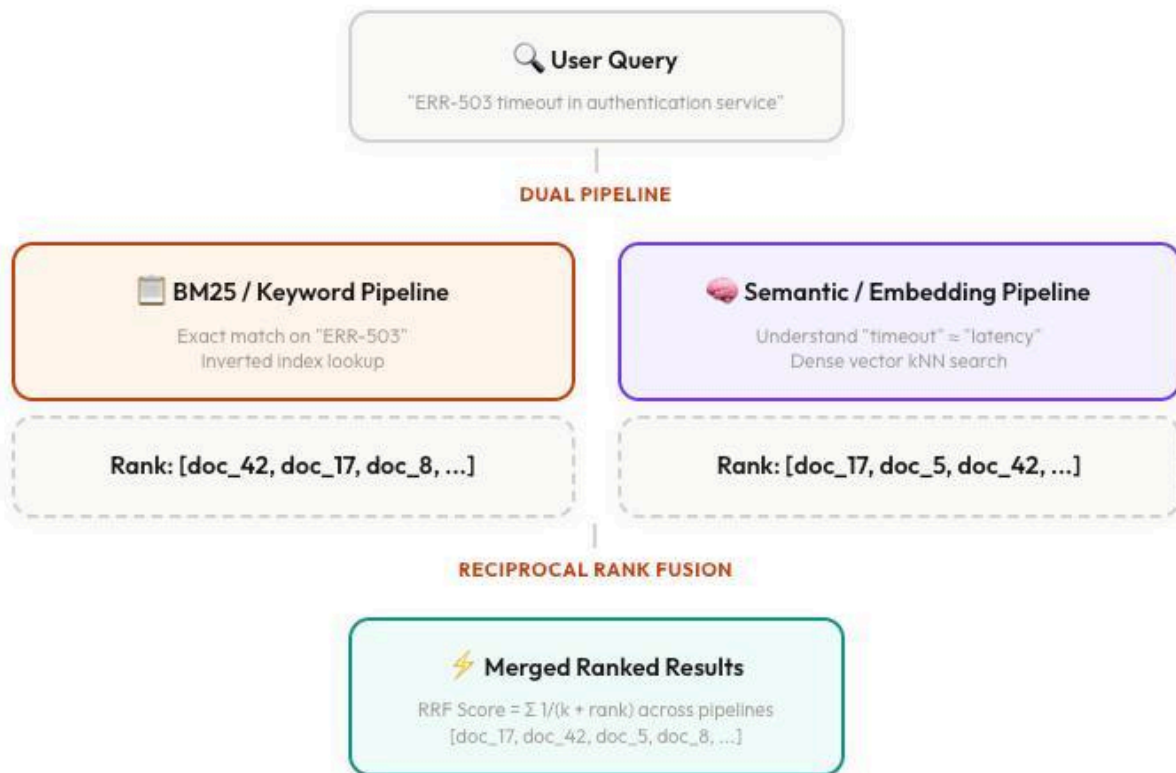
This gives us a meaningful experiment to include in the report.

3. Overall system architecture

Our system will have five major layers.

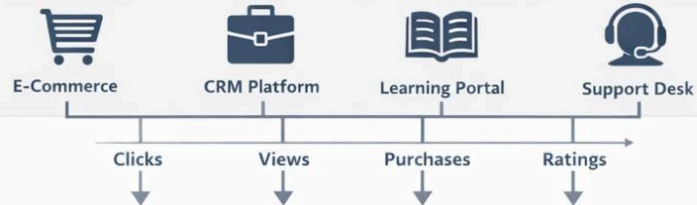


HYBRID SEARCH ARCHITECTURE



Recommendation System Architecture

User Interaction Layer



Data Processing Layer



Recommendation Engine

Content-Based Filtering

Collaborative Filtering

Hybrid Model
← Business Rules

- Similarity Matching
- User Behavior Analysis
- Real-Time Context

Output & Feedback Loop



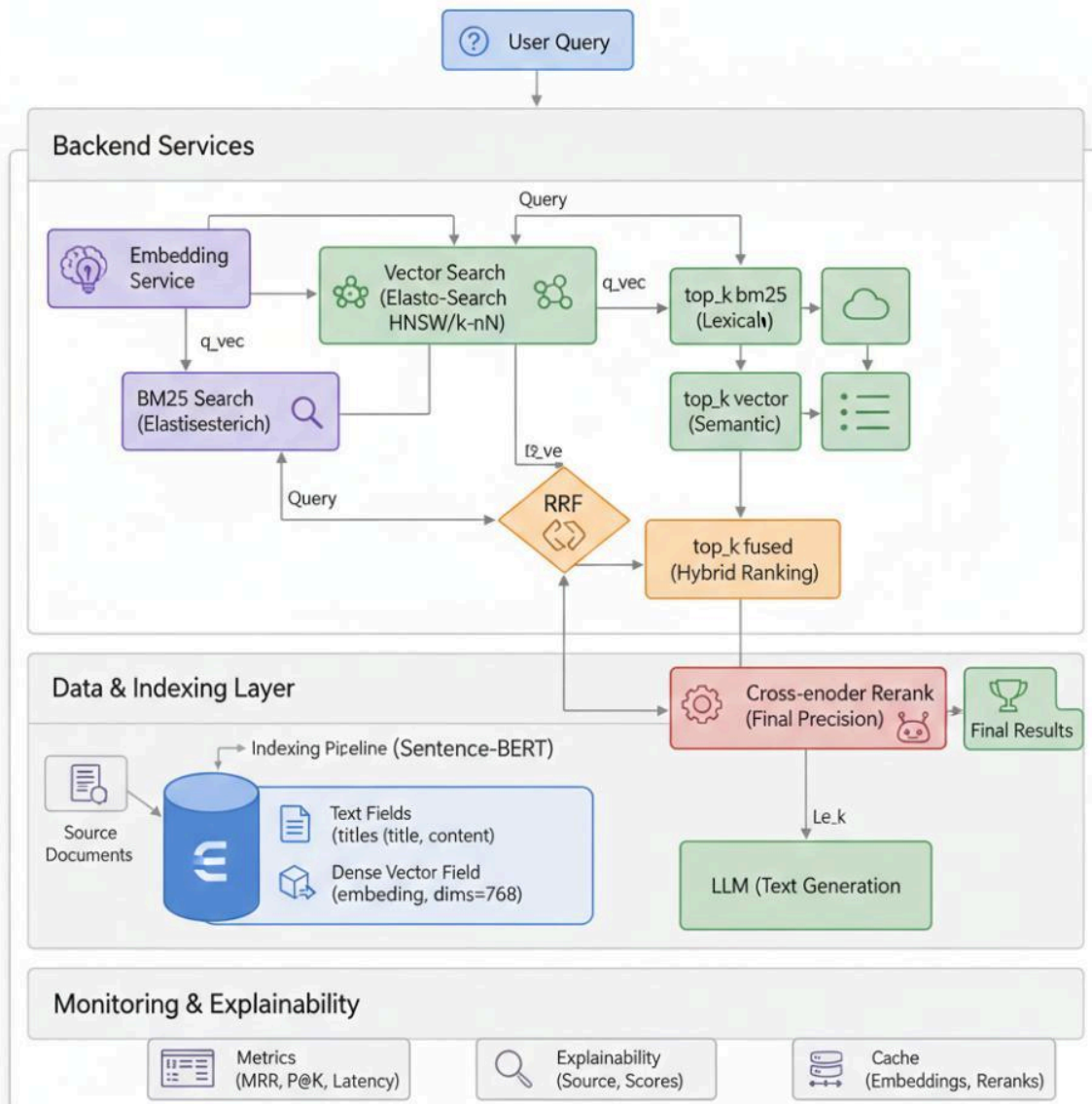
Personalized Recommendations



Analytics & Reports

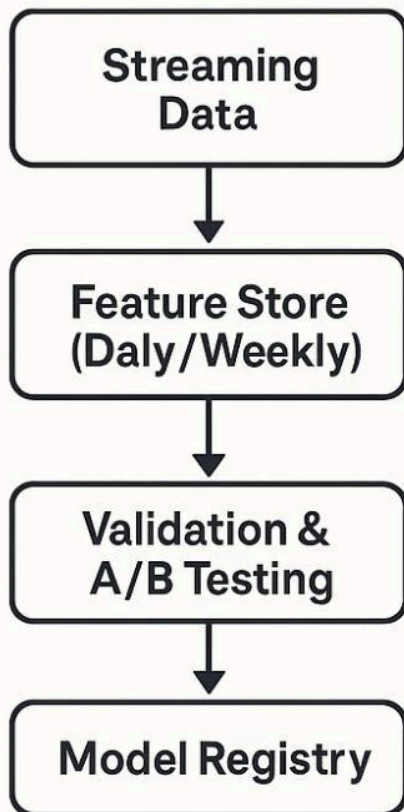
Feedback Data

- Performance Metrics
- User Interactions

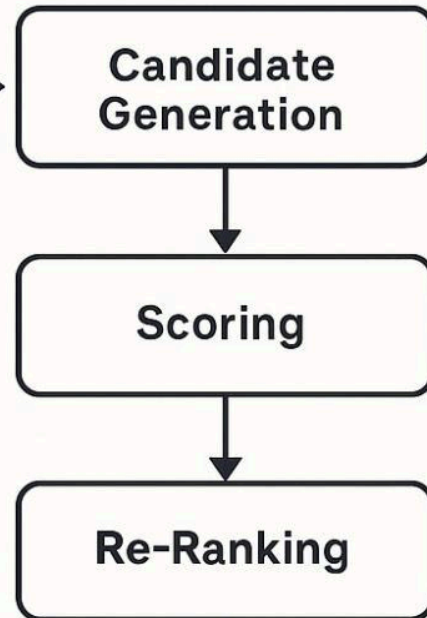


REAL-TIME RECOMMENDATION SYSTEM

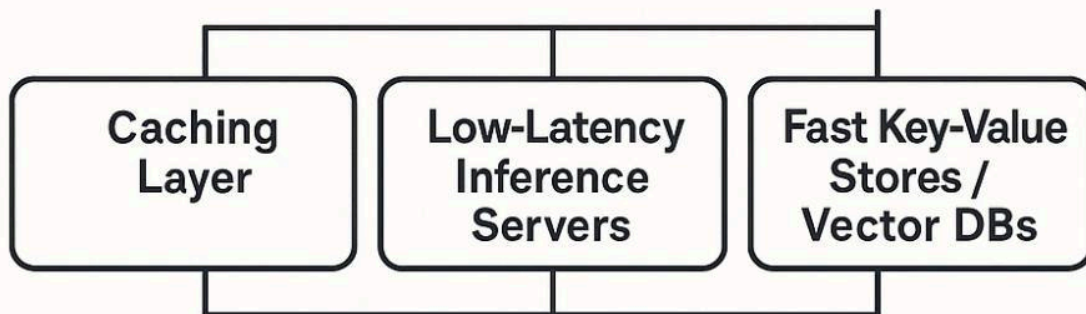
CONTINUOUS TRAINING PIPELINE



SERVING INFRASTRUCTURE



SUPPORTING INFRASTRUCTURE



SUPPORTING INFRASTRUCTURE

Layer 1: Frontend

The React application will provide the interface through which students interact with the system.

It will contain:

- Course discovery
- Search results
- Recommendation explanations
- Skill gap analysis
- Learning path visualization
- What-If interaction
- Evaluation dashboard
- Feedback

Layer 2: API backend

FastAPI will connect the frontend to the recommendation engine.

It will handle:

- Search requests
- User profiles
- Recommendation requests
- Skill analysis
- Learning path generation
- Feedback submission
- Evaluation requests

Layer 3: Recommendation engine

This is the central component of the project.

It will contain:

- BM25 search
- Semantic search
- Hybrid ranking
- Personalized ranking
- Alternative course selection
- Recommendation explanations

Layer 4: Skill analysis and learning path

This layer will handle:

- Skill extraction
- Skill normalization
- Skill gap identification
- Prerequisite relationships
- Learning path generation
- Skill graph construction
- What-If simulations

Layer 5: Data and evaluation

This layer will contain:

- Processed course data
- Course embeddings
- Skill dependency information
- Evaluation queries
- Relevance judgments
- User feedback

The evaluation system will use the same search and recommendation components as the main application.

This is important because we want to evaluate the actual system rather than a separate implementation.

4. Phase 1: Dataset preparation and knowledge base

Assignment weightage: 20%

This is where we'll begin implementation.

We have two CSV files:

Dataset	Row s	Column s	Role
<code>coursera_course_dataset_v3.csv</code>	623	12	Primary dataset

`coursera_course_dataset_v2_no_n` 623 7
`ull.csv`

Supplementary
dataset

We'll use v3 as our primary source because it contains course descriptions, URLs, difficulty, type, and duration.

The v2 dataset will be useful for cross-checking metadata.

4.1 Data inspection

First, we'll create a data inspection script.

It will check:

- Number of courses
- Column names and data types
- Missing values
- Duplicate courses
- Unique organizations
- Difficulty distribution
- Rating distribution
- Number of courses with descriptions
- Number of courses with skills
- Number of courses with URLs

We'll save a summary of these results.

This will help us document the original dataset and justify our preprocessing decisions.

4.2 Data cleaning

We'll standardize the dataset.

Planned operations:

1. Remove unnecessary columns such as `Unnamed: 0`.
2. Standardize column names.
3. Remove exact duplicate records.
4. Normalize whitespace and text encoding.
5. Convert ratings and enrollment figures into appropriate numeric formats.
6. Standardize difficulty levels.
7. Standardize course types and durations.
8. Handle missing values.

We won't blindly remove courses with missing descriptions.

If a course has a title and a skills list, it may still be useful for keyword search and recommendations.

4.3 Description validation

During our initial inspection, we found some descriptions that appear unrelated to their course titles.

This is a significant data quality problem.

For example, a course titled Google Cybersecurity has a description about Google Cloud Fundamentals.

We should identify these cases before generating embeddings.

Our validation process will:

1. Compare course titles with their descriptions.
2. Identify suspicious title-description combinations.
3. Review the flagged records.
4. Mark questionable descriptions.
5. Exclude unreliable descriptions from semantic retrieval where necessary.

We can use a combination of automated checks and manual review.

We should not automatically delete a course just because its description is unusual.

4.4 Skill normalization

The dataset contains skills such as:

- Python Programming
- Computer Programming
- Data Analysis
- Data Visualization
- Machine Learning

Some skill names may overlap or refer to related concepts.

We'll create a skill normalization module.

For example:

Original skill	Normalized skill
Python Programming	Python
Programming in Python	Python
Data Visualisation	Data Visualization
ML	Machine Learning

We'll use a controlled mapping rather than merging every similar-sounding skill.

For instance, Python and general programming should remain separate skills.

4.5 Course knowledge base

After cleaning, we'll produce a standardized course representation.

An example:

```
{
  "course_id": 1,
  "title": "Example Machine Learning Course",
  "organization": "Example University",
  "description": "Introduction to machine learning...",
  "skills": [
    "Python",
    "Machine Learning",
    "Data Analysis"
  ],
  "difficulty": "Intermediate",
  "rating": 4.7,
  "duration": "3 Months",
  "course_url": "https://example.com/course",
  "description_quality": "verified"
}
```

This is an illustrative record, not an actual dataset entry.

Every course will have a stable internal ID.

We'll also retain the original data for traceability.

4.6 Search index creation

We'll prepare two search representations for each course.

BM25 document

A combined text field containing:

- Course title
- Course description
- Skills
- Organization
- Course type

Semantic document

A structured text representation emphasizing:

- Course title
- Course description
- Skills
- Difficulty

We'll experiment with the text composition to see which fields improve retrieval.

We should avoid letting generic words from organizations or course types overwhelm the actual course content.

4.7 Embedding generation

We'll use a local Sentence Transformers model to generate embeddings.

Each course will have a vector representation.

We'll initially generate embeddings for the available, validated course text.

For courses with missing or unreliable descriptions, we'll use their titles and skills where appropriate.

The embeddings will be stored locally so that we don't need to regenerate them every time the application starts.

With just 623 courses, we can initially perform cosine similarity calculations directly using NumPy. We don't need a complicated vector database yet.

Phase 1 deliverables

By the end of this phase, we'll have:

- A cleaned course dataset
- A data validation report
- A normalized skills list
- A course knowledge base
- BM25 index
- Semantic embeddings
- A preprocessing script

Completion condition: We can load the processed dataset and retrieve course records reliably.

5. Phase 2: Hybrid search and recommendation engine

Assignment weightage: 25%

This is the core of the project.

We'll implement the search engine independently of the frontend so that it can be tested through the API and command line.

5.1 Natural-language query processing

Students should be able to enter queries such as:

- "I want to learn data science from scratch."
- "I know Python and SQL. What should I learn next?"
- "I want to get into cloud computing."
- "Suggest courses for someone interested in cybersecurity."

We'll initially support direct queries without requiring an LLM.

We can extract useful information using:

- Keyword detection
- Skill matching
- Basic intent classification
- Optional LLM-assisted query refinement

The system should distinguish between a learning goal and a student's existing skills whenever possible.

For example:

"I know Python and want to learn machine learning."

The system should identify:

Field	Value
Goal	Machine learning
Existing skill	Python
Difficulty preference	Not specified
Career goal	Not specified

We shouldn't assume information that the student hasn't provided.

5.2 BM25 search

We'll implement BM25 using a suitable Python library.

It will retrieve courses based on lexical relevance.

The search will consider:

- Course titles
- Skills
- Descriptions

We can assign different importance to fields if our experiments show that doing so improves retrieval.

For example, matching a course title may be more useful than matching a generic word buried in its description.

5.3 Semantic search

We'll use Sentence Transformers to encode the student's query.

Then we'll compare the query embedding with the stored course embeddings using cosine similarity.

This should help with queries that use different wording from the course descriptions.

For example, a student searching for "how to build intelligent computer programs" might be interested in machine learning courses even if the exact phrase isn't present.

However, semantic search can also retrieve conceptually related but unsuitable courses. That's why we'll combine it with lexical search and later ranking.

5.4 Hybrid retrieval

We'll implement RRF as our initial hybrid retrieval method.

The process will be:

1. Retrieve the top BM25 results.
2. Retrieve the top semantic results.
3. Combine their rankings using RRF.
4. Remove duplicates.
5. Return the highest-ranked courses.

We'll retain the individual rankings for evaluation and debugging.

We can later compare RRF against normalized weighted scores.

5.5 Personalized ranking

Retrieval finds relevant courses, but recommendation requires more than relevance.

We'll add a separate ranking stage.

Possible ranking factors:

Factor	Purpose
Hybrid relevance	Match the student's goal
Skill coverage	Prefer courses that teach missing skills
Difficulty compatibility	Account for the student's experience
Rating	Provide an additional quality signal
Existing skill overlap	Identify courses that build on known skills

Course availability

Avoid recommending unusable or incomplete entries

We'll start with a simple, configurable ranking formula.

The exact weights will be determined through evaluation, rather than selected arbitrarily and presented as optimal.

We should also make sure that ratings don't dominate the relevance of a course.

A highly rated course that doesn't match the student's goal shouldn't automatically outrank a relevant one.

5.6 Filters

The search page will support filters for:

- Difficulty
- Organization
- Course type
- Rating
- Duration

We'll also consider sorting by relevance, rating, and duration.

Phase 2 deliverables

- BM25 search
- Semantic search
- Hybrid retrieval
- Personalized ranking
- Search filters
- Search API
- Search evaluation baseline

Completion condition: Given a natural-language query, the system returns relevant courses with consistent rankings and working filters.

6. Phase 3: Skill gap analysis and learning path generation

Assignment weightage: 25%

This phase is especially important because it distinguishes our project from a conventional course search application.

6.1 Student profile

We'll create a student profile containing:

```
{  
  "learning_goal": "Machine Learning",  
  "existing_skills": [  
    "Python",  
    "SQL"  
  ],  
  "difficulty_preference": "Beginner",  
  "career_goal": "Machine Learning Engineer"  
}
```

Students should be able to enter their skills manually.

We'll also allow them to add or remove skills later.

For the initial version, we don't need user accounts. We can manage the current session's profile in the frontend.

If we add persistent profiles later, we'll use SQLite.

6.2 Skill extraction

We'll extract skills from:

- The student's query
- The student's manually entered profile
- The course dataset

We'll normalize them using the mapping created in Phase 1.

We should allow users to correct extracted skills because automatic extraction won't always be accurate.

6.3 Skill requirement identification

For each learning goal, we'll identify the skills associated with that goal.

For example:

Goal: Machine learning

Potential skills:

- Python
- Statistics
- Data preprocessing
- Machine learning
- Model evaluation

We can build an initial skill taxonomy for several supported areas, such as:

- Data science
- Machine learning
- Cybersecurity
- Cloud computing
- Web development
- Data analytics

The taxonomy will be based on skills actually present in the dataset.

We'll expand it only where the data supports useful recommendations.

6.4 Skill gap analysis

We'll compare the student's existing skills with the target skills.

For example:

Skill	Status
Python	Already known
SQL	Already known
Statistics	Missing
Data preprocessing	Missing
Machine learning	Missing
Model evaluation	Missing

The system will then find courses that cover the missing skills.

A course that covers several missing skills may be particularly useful.

We'll avoid treating every missing skill as equally important if we have sufficient information to distinguish foundational skills from optional ones.

6.5 Prerequisite relationships

This is one of the more challenging parts of the project.

The dataset doesn't provide a reliable prerequisite graph.

We'll therefore create a separate skill dependency model.

For example:

```
Python
|
v
Data Preprocessing
|
v
Machine Learning
|
v
Model Evaluation
```

These are illustrative relationships.

We'll explicitly mark whether a relationship is:

- Provided by course metadata
- Derived from a manually defined skill dependency
- Inferred from course descriptions

This prevents the application from presenting guesses as official requirements.

We should also allow courses to have no known prerequisites.

6.6 Learning path generation

We'll generate a learning path by considering:

1. The student's existing skills

2. The target skills
3. Course difficulty
4. Skill dependencies
5. Course coverage
6. Course relevance

The system should prefer a logical progression from foundational to advanced material.

For example:

Step	Course type	Purpose
1	Beginner	Learn foundational concepts
2	Beginner/Intermediate	Build supporting skills
3	Intermediate	Learn the main subject
4	Advanced	Develop specialized knowledge

The actual courses will be selected from the dataset.

We'll avoid recommending a course solely because its difficulty is marked Beginner. Its skills and relevance must also match.

If the dataset doesn't contain an appropriate course for a particular skill, the system should report that gap instead of inventing a course.

6.7 Alternative courses

For every recommended course, we'll identify alternatives using:

- Similarity to the original course
- Overlapping skills
- Similar difficulty
- Similar learning goals

The alternatives should be ranked separately from the main learning path.

We'll explain why each alternative is similar and how it differs.

Phase 3 deliverables

- Student profile

- Skill extraction and normalization
- Skill gap analyzer
- Skill dependency model
- Learning path generator
- Alternative course selector
- Explanation engine

Completion condition: Given a student profile and learning goal, the system produces a reasonable set of missing skills and an ordered learning path, with explanations for its decisions.

7. Phase 4: Interactive features

These features will make the application more engaging and help fulfill your staff's request for creativity.

We'll implement them on top of the skill analysis and recommendation engine.

7.1 Interactive Skill Graph

We'll use React Flow to visualize the student's skills and learning path.

The graph will include:

- Existing skills
- Missing skills
- Recommended courses
- Skill dependencies
- Target learning goal

Students will be able to click nodes to see more information.

For example, selecting a missing skill could display courses that teach it.

The graph will be generated using structured data from the backend.

It won't independently decide which skills depend on one another.

7.2 What-If Skill Toggle

We'll add a panel where students can simulate learning additional skills.

For example:

Current skills

- Python
- SQL

Skills to simulate

- Statistics
- Data preprocessing
- Machine learning

Students can toggle any of these skills.

When a skill is selected:

1. The frontend updates the simulated profile.
2. The backend recalculates the skill gaps.
3. The learning path is updated.
4. The recommendations are refreshed.
5. The skill graph changes accordingly.

We'll distinguish simulated skills from skills the student has actually confirmed.

This is important because the interface should not permanently alter the student's real profile when they are only experimenting.

7.3 Honest Advisor

Each recommendation card will have an expandable explanation panel.

It will display:

Why recommended

- Relevance to the learning goal
- Matching skills
- Skills the course can help develop

Potential concerns

- Difficulty mismatch
- Missing prerequisite skills
- Limited relevance to the stated goal
- Missing course information

Alternatives

- Similar courses
- Courses with a more suitable difficulty
- Courses covering overlapping skills

We'll implement these explanations using the recommendation engine's actual results and rules.

An optional LLM can make the wording more conversational, but it won't be responsible for deciding the facts.

7.4 Course comparison

This is an optional additional feature I'd like to include if time permits.

Students could select two or more courses and compare:

- Skills covered
- Difficulty
- Rating
- Duration
- Organization
- Relevance to their goal

We could also highlight differences in course content.

This is useful, but it comes after the core features.

8. Phase 5: Evaluation and personalization

Assignment weightage: 20%

We'll develop the evaluation system alongside the search engine, then integrate it into the frontend.

This phase should not be left until the end.

8.1 Evaluation dataset

We'll create a collection of representative student queries.

For example:

Query	Existing skills	Goal
Learn data science from scratch	None	Data science
Move from Python to machine learning	Python	Machine learning
Learn cloud computing	None	Cloud computing
Advance from SQL to data analytics	SQL	Data analytics
Learn cybersecurity fundamentals	None	Cybersecurity

For each query, we'll manually identify relevant courses from the dataset.

This will serve as our initial relevance judgment set.

We should include queries with different levels of specificity.

8.2 Search evaluation

We'll compare:

- BM25
- Semantic search
- Hybrid search using RRF
- Optionally, hybrid search using weighted scores

We'll use metrics such as:

Precision@K

Measures the proportion of the top K results that are relevant.

Recall@K

Measures the proportion of known relevant courses retrieved in the top K results.

Hit Rate@K

Measures whether at least one relevant course appears in the top K results.

MRR

Measures how highly the first relevant result appears.

We'll calculate these metrics using our manually reviewed relevance judgments.

8.3 Skill gap evaluation

We'll prepare test cases with expected skill gaps.

For each case, we'll compare the system's output with the expected missing skills.

We'll report suitable metrics such as precision, recall, and F1-score for skill identification.

8.4 Learning path evaluation

Learning paths are more difficult to evaluate automatically.

We'll create representative test cases and manually review:

- Whether the course order is logical
- Whether required foundational skills appear before advanced ones
- Whether the courses cover the target skills
- Whether the path contains unnecessary courses

We can also measure skill coverage and the number of dependency violations.

8.5 Performance evaluation

We'll measure:

- Search latency
- Recommendation latency
- Embedding generation time
- Memory consumption, if practical
- API response times

We'll report the hardware and test conditions used.

8.6 Feedback collection

Students should be able to provide feedback on recommendations.

For example:

- Relevant
- Not relevant

- Too advanced
- Too basic
- Already learned
- Other

We'll store feedback in SQLite.

Initially, feedback will be used for analysis rather than automatically changing the recommendation algorithm.

Once we have enough feedback, we can consider using it to improve personalization.

Phase 5 deliverables

- Evaluation query set
- Relevance judgments
- Search metrics
- Skill gap test cases
- Learning path test cases
- Performance measurements
- Feedback storage
- Evaluation dashboard

Completion condition: We can run repeatable tests and show measurable results for the recommendation system.

9. Phase 6: React frontend

Although frontend development is listed later here, we'll build it incrementally alongside the backend.

The final application should feel like a real product rather than a collection of disconnected forms.

Proposed UI structure

Page	Main components
Dashboard	Search bar, learning goal input, profile summary

Discover Courses	Filters, search results, sorting
Recommendations	Course cards, explanations, alternatives
Skill Analysis	Existing skills, missing skills, skill coverage
Learning Path	Course sequence, dependencies, progress
Skill Graph	Interactive dependency graph
What-If	Skill toggles and dynamic recalculation
Evaluation	Search comparison, metrics, latency
Feedback	Recommendation ratings and comments

We can organize these into a sidebar with a consistent layout.

Suggested design

I recommend:

- A clean dashboard
- A consistent navigation sidebar
- Search results in cards
- Expandable explanations
- Interactive charts
- Clear loading and error states
- Responsive layouts

We can use a visual theme appropriate for an educational technology application.

However, we should prioritize readability and usability over excessive animation or decorative elements.

Frontend-backend integration

React will communicate with FastAPI through REST endpoints.

We'll define the API contracts before integrating each page.

This will let us develop and test the frontend and backend independently.

10. Phase 7: API fallback and reliability

We'll implement reliability features after the core search and recommendation pipeline works.

The main principle is that **the core system must not depend on an external AI API**.

Normal operation

1. Student submits a query.
2. Backend performs BM25 and semantic search.
3. Recommendation engine ranks courses.
4. Skill analysis and learning path generation run.
5. The explanation engine creates evidence-based explanations.
6. The frontend displays the results.

If the optional AI API fails

The system will:

- Log the failure.
- Use local query processing.
- Continue retrieval and ranking.
- Generate template-based explanations.
- Return the results normally.

We can also implement:

- Request timeouts
- Limited retries
- Error logging
- Health checks
- Clear API status indicators

We don't need a secondary API key initially.

A local embedding model and rule-based explanation engine are sufficient for a meaningful fallback.

We'll also include a way to simulate API failure during testing.

11. Phase 8: Backend API design

We'll keep the backend modular and expose its functionality through FastAPI.

Here's the proposed API structure.

Endpoint	Method	Purpose
<code>/health</code>	GET	Check backend status
<code>/courses</code>	GET	Browse available courses
<code>/courses/{id}</code>	GET	Retrieve course details
<code>/search</code>	POST	Perform keyword, semantic, or hybrid search
<code>/recommend</code>	POST	Generate personalized recommendations
<code>/skills/analyze</code>	POST	Analyze skill gaps
<code>/learning-path</code>	POST	Generate a learning path
<code>/learning-path/simulate</code>	POST	Recalculate a path for What-If skills
<code>/courses/compare</code>	POST	Compare selected courses
<code>/feedback</code>	POST	Submit recommendation feedback
<code>/evaluation/run</code>	POST	Run evaluation tests
<code>/evaluation/results</code>	GET	Retrieve evaluation results

These are proposed endpoints. We'll refine their request and response schemas when we start implementation.

For example, `/recommend` could accept:

```
{
  "goal": "Learn machine learning",
  "existing_skills": [
    "Python",
    "SQL"
  ],
  "difficulty": "Beginner",
  "top_k": 5
}
```

```
}
```

And return:

- Recommended courses
- Relevance scores
- Matching skills
- Missing skills
- Explanations
- Alternatives
- Learning path

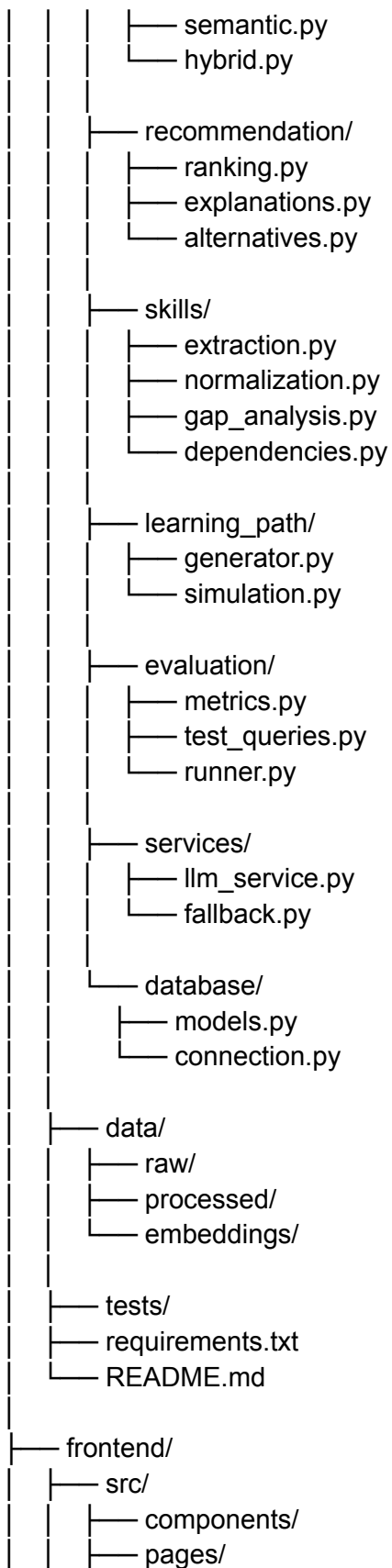
We should keep API responses consistent and include appropriate error messages.

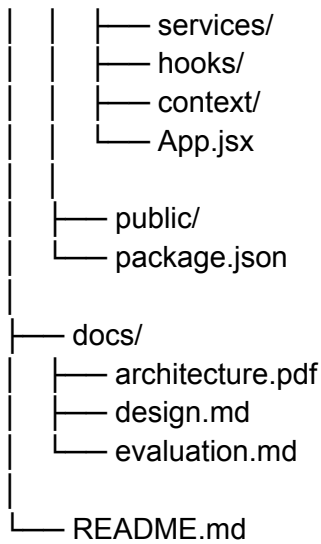
12. Phase 9: Project structure

I recommend keeping the frontend and backend in separate directories.

Our initial project structure could look like this:

```
university-course-finder/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── courses.py
│   │   │   ├── search.py
│   │   │   ├── recommendations.py
│   │   │   ├── skills.py
│   │   │   ├── learning_path.py
│   │   │   ├── evaluation.py
│   │   │   └── feedback.py
│   │   ├── data/
│   │   │   ├── preprocessing.py
│   │   │   ├── validation.py
│   │   │   └── knowledge_base.py
│   │   └── search/
│   │       └── bm25.py
```





This is a target structure, not something we need to create all at once.

We'll add modules only when we need them.

13. Testing strategy

We should test every major component independently before integrating the complete application.

Component	Tests
Data preprocessing	Missing values, duplicates, incorrect formats
Description validation	Flag suspicious course descriptions
Skill normalization	Map equivalent terms correctly
BM25	Retrieve courses matching relevant keywords
Semantic search	Retrieve courses related to the query's meaning
Hybrid search	Combine results without duplicates
Recommendation ranking	Apply preferences consistently
Skill gap analysis	Identify missing skills accurately

Learning path	Respect known dependencies
What-If simulation	Update results when skills change
API	Validate requests and responses
Fallback	Continue functioning when the AI API fails
Frontend	Test forms, filters, and interactions

We'll also perform end-to-end tests using complete student scenarios.

For example:

Scenario: A student knows Python and wants to learn machine learning.

We should verify that:

1. The query returns relevant courses.
2. Existing skills are recognized.
3. Missing skills are identified.
4. Recommended courses cover those skills.
5. The learning path is logically ordered.
6. The explanations accurately reflect the results.
7. The What-If feature updates the path.
8. Feedback can be submitted.

This will be useful for the final demonstration.

14. Documentation and final deliverables

Assignment weightage: 10%

We'll prepare the required documentation as we develop the project instead of writing everything at the last minute.

Architecture diagram

The diagram should show:

- Raw dataset
- Data preprocessing
- Knowledge base

- BM25 indexing
- Embedding generation
- Search and ranking
- Skill gap analysis
- Learning path generation
- API
- React frontend

We'll export it as a PDF or JPEG.

Technical design document

It should explain:

1. System architecture
2. Dataset characteristics
3. Data preprocessing decisions
4. Search algorithms
5. Hybrid retrieval approach
6. Recommendation ranking
7. Skill gap analysis
8. Prerequisite modeling
9. Learning path generation
10. API fallback
11. Evaluation methodology
12. Technical trade-offs

README

The README should include:

- Project overview
- Features
- Technologies
- Installation
- Configuration
- How to run the backend
- How to run the frontend
- Sample API requests
- Example student queries
- Evaluation instructions

Final presentation

For your 10-minute presentation, I'd suggest:

Time	Content
1 minute	Problem statement and motivation
1 minute	Architecture and technical approach
2 minutes	Hybrid search and recommendations
2 minutes	Skill gap analysis and learning path
1 minute	Skill graph and What-If interaction
1 minute	Evaluation and fallback demonstration
2 minutes	Questions and answers

We can adjust this depending on your panel's expectations.

15. Recommended implementation order

Although the project has nine phases, I recommend implementing them in the following order.

Order	Task	Priority
1	Dataset inspection and cleaning	Critical
2	Skill normalization and knowledge base	Critical
3	BM25 search	Critical
4	Semantic search	Critical
5	Hybrid retrieval and ranking	Critical
6	Initial evaluation dataset	Critical
7	Skill gap analysis	Critical
8	Learning path generation	Critical
9	Recommendation explanations and alternatives	Critical
10	FastAPI integration	Critical

11	Basic React interface	Critical
12	Evaluation dashboard	Critical
13	Feedback collection	High
14	Interactive skill graph	High
15	What-If Skill Toggle	High
16	Optional LLM integration and fallback	High
17	Course comparison	Optional
18	Final UI polish and documentation	Critical

One adjustment: we'll build the API and frontend incrementally rather than waiting until all recommendation features are complete. That way, we can test the system through the interface as we develop it.

16. Definition of a completed project

Before we consider the project finished, I suggest we use the following checklist.

Data

- Both supplied CSV files inspected.
- Primary dataset cleaned.
- Missing values handled.
- Suspicious descriptions reviewed.
- Skills normalized.
- Search index generated.
- Embeddings generated and saved.

Search and recommendations

- BM25 implemented.
- Semantic search implemented.
- Hybrid retrieval implemented.
- Search filters working.
- Personalized ranking working.
- Recommendation explanations available.

- Alternatives available.

Skill analysis

- Student profile supported.
- Skill extraction implemented.
- Skill gap analysis working.
- Prerequisite model implemented.
- Learning path generation working.
- Skill graph displayed.
- What-If simulation working.

Application

- React frontend connected to FastAPI.
- Search interface working.
- Recommendation cards displayed.
- Skill analysis page working.
- Learning path page working.
- Evaluation page working.
- Feedback stored successfully.
- API fallback tested.

Evaluation and documentation

- Representative queries created.
- Relevance judgments prepared.
- Retrieval metrics calculated.
- Skill gap tests completed.
- Learning paths reviewed.
- Architecture diagram created.
- README completed.
- Final presentation prepared.

17. What we should actually build first

I recommend that we divide the implementation into smaller milestones.

Milestone 1: Data foundation

- Inspect both CSVs.

- Finalize the canonical course schema.
- Build the preprocessing pipeline.
- Validate the data.
- Create the first processed dataset.

Milestone 2: Search foundation

- Implement BM25.
- Implement semantic search.
- Implement RRF.
- Test the three retrieval methods.
- Create our initial evaluation queries.

Milestone 3: Recommendation intelligence

- Build personalized ranking.
- Implement skill gap analysis.
- Define the initial skill dependency model.
- Generate learning paths.
- Implement explanations and alternatives.

Milestone 4: Application

- Build the FastAPI endpoints.
- Create the React layout.
- Connect the search and recommendation features.
- Add the evaluation dashboard.

Milestone 5: Interactive features

- Add the skill graph.
- Implement What-If simulations.
- Add feedback.
- Add the optional LLM and fallback.

Milestone 6: Finalization

- Run tests.
- Tune the retrieval and ranking methods.
- Fix UI issues.
- Complete the documentation.
- Prepare the demonstration.

This sequence gives us a working core before we invest time in more elaborate features.

18. My final recommendations

There are five decisions I think we should keep throughout the project.

1. Use BM25 and semantic search together.

Keep TF-IDF as a baseline for comparison, rather than making it the main lexical search method.

2. Keep the recommendation engine independent of the LLM.

The search, ranking, skill analysis, and learning path generation should work locally.

3. Build one shared skill model.

The skill graph, What-If feature, learning path generator, and Honest Advisor should all use the same skill relationships and student profile.

4. Evaluate before claiming improvement.

We'll compare retrieval methods and test recommendation quality instead of assuming that hybrid search is automatically better.

5. Prioritize correctness over the number of features.

A reliable recommendation system with clear explanations is more valuable than a visually impressive application that produces inconsistent recommendations.

One practical consideration: the dataset contains only 623 courses. That's perfectly workable for this project, but the quality of our results will depend heavily on cleaning and interpreting those records correctly. We should be transparent about that limitation in the final report.

Our immediate next step: Milestone 1

We'll begin with the dataset, not the UI.

Our first implementation task will be to create a **data inspection and preprocessing pipeline**.

We'll do it in this order:

1. Inspect the columns and data types.
2. Compare the two datasets.
3. Identify missing and suspicious records.
4. Decide how to handle each type of data issue.
5. Create the canonical course schema.
6. Implement the cleaning script.
7. Generate the processed dataset.

8. Validate the output.

Once that is complete, we'll have a reliable foundation for the search engine.

This is the implementation plan I'd use for the project. It covers all five assignment tasks, incorporates your staff's feedback, and gives us a clear distinction between essential features and optional enhancements. We can now start Milestone 1 and work through the implementation one component at a time.